



nextwork.org

Deploy Backend with Kubernetes

CH

Arpita Chowdhury

```
[ec2-user@ip-172-31-25-198 manifests]$ sudo curl -o /usr/local/bin/kubectl \
https://s3.us-west-2.amazonaws.com/amazon-eks/1.31.0/2024-09-12/bin/linux/amd64/kubectl
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 53.7M 100 53.7M 0 9511k 0 0:00:05 0:00:05 --:--:-- 10.2M
[ec2-user@ip-172-31-25-198 manifests]$ sudo chmod +x /usr/local/bin/kubectl
[ec2-user@ip-172-31-25-198 manifests]$ kubectl version
Client Version: v1.31.0-eks-a737599
Kustomize Version: v5.4.2
Server Version: v1.33.6-eks-5afda5c
WARNING: version difference between client (1.31) and server (1.33) exceeds the supported minor version skew of +/-1
[ec2-user@ip-172-31-25-198 manifests]$ kubectl apply -f flask-deployment.yaml
deployment.apps/nextwork-flask-backend created
[ec2-user@ip-172-31-25-198 manifests]$
```



Introducing Today's Project!

In this project, I will install and configure kubectl, prepare my backend for deployment, and deploy it onto my Amazon EKS Kubernetes cluster. I am doing this because kubectl is the primary tool used to interact with Kubernetes, and deploying the backend through it allows me to manage, monitor, and control my application in a real production-style Kubernetes environment. This gives me hands-on experience with how cloud-native applications are actually launched and maintained in modern infrastructure.

Tools and concepts

I used Kubernetes, Amazon ECR, kubectl, and Amazon EKS to deploy a containerized backend application into a cloud-native production environment. Amazon ECR stored the Docker image, Kubernetes handled orchestration, and kubectl was used to apply manifests and manage the running application. Key concepts include using Kubernetes manifests to declaratively define how applications should run, scale, and be exposed, including Deployments for managing pods and Services for routing traffic, allowing Kubernetes to automatically maintain the desired state of the system.

Project reflection

This project took me approximately 2 hours to complete, including building the container image, creating Kubernetes manifests, and deploying the backend to the EKS cluster. The most challenging part was understanding how all the pieces fit together especially how ECR, Kubernetes, Deployments, Pods, and Services interact to deliver a running application. My favourite part was watching Kubernetes automatically pull the image from ECR and start the containers, which made the deployment feel truly production grade.



Project Set Up

Kubernetes cluster

To set up today's project, I launched a Kubernetes cluster. Steps I took to do this included running this command: `eksctl create cluster \ --name nextwork-eks-cluster \ --nodegroup-name nextwork-nodegroup \ --node-type t3.micro \ --nodes 3 \ --nodes-min 1 \ --nodes-max 3 \ --version 1.33 \ --region us-east-1` in my Ec2 instance terminal I created for this project.

Backend code

I retrieved the backend that I plan to deploy. An app's backend is the server-side part of the application that handles requests, runs business logic, and communicates with data sources. It is responsible for processing user actions and returning the correct responses to the frontend.

Container image

Once I cloned the backend code, I built a container image because Kubernetes requires applications to be packaged into container images before they can be deployed and scaled across a cluster. I used Docker and the project's Dockerfile to build the image, which installed all the required dependencies, copied the backend code into the container, and defined how the Flask application should start when the container runs.

A container registry is a centralized storage service where container images are stored, versioned, and made available for deployment by container platforms like Kubernetes. ECR facilitates scaling for my deployment because it allows Kubernetes to quickly pull the same container image whenever new Pods need to be created. When traffic increases and Kubernetes scales the application, it can instantly download the image from ECR and launch more identical containers, ensuring fast, consistent, and reliable scaling across the cluster.



Manifest files

In this step, I will create a Kubernetes Service manifest that defines how network traffic should be routed to my backend application running in the EKS cluster. I am doing this because a Deployment only ensures that containers are running, but it does not make them reachable. A Service provides a stable network endpoint and automatically load-balances traffic across all running pods, allowing users and other services to reliably access the backend application.

A Deployment manifest manages how your application is run in Kubernetes by controlling how many Pods are created, keeping them healthy, replacing failed Pods, and handling updates to the application in a controlled way. The container image URL in my Deployment manifest tells Kubernetes exactly where to pull the Docker image from (in this case, Amazon ECR), so it knows which version of the backend application to run when creating or scaling Pods.

A Service resource exposes a group of Pods through a single, stable network endpoint and automatically load-balances traffic between them, even as Pods are created, deleted, or replaced. My Service manifest sets up a NodePort service that exposes the Flask backend on port 8080 and routes incoming requests to all Pods labeled `app: nextwork-flask-backend`, making the application accessible from outside the EKS cluster.



```
GNU nano 8.3 flask-deployment.yaml
--
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextwork-flask-backend
  template:
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend
          image: 006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest
          ports:
            - containerPort: 8080
```



Backend Deployment!

To deploy my backend application, I installed kubectl, connected it to my EKS cluster, and used it to apply my Kubernetes Deployment manifest (flask-deployment.yaml), which instructed Kubernetes to pull my container image from Amazon ECR and create the backend Pods inside the cluster.

kubectl

kubectl is the official Kubernetes command-line tool used to communicate with the Kubernetes API server and manage resources inside a cluster (like Pods, Deployments, and Services). I need this tool to deploy my application by applying my YAML manifests, and to verify and troubleshoot the deployment (e.g., check pods/nodes, view services, inspect logs, and describe resources). I can't use eksctl for the job because eksctl is mainly for provisioning and managing the EKS infrastructure (clusters, node groups, IAM/OIDC), while kubectl is the tool designed to manage what runs inside the cluster Kubernetes workloads and networking objects.



```
[ec2-user@ip-172-31-25-198 manifests]$ sudo curl -o /usr/local/bin/kubect1 \
https://s3.us-west-2.amazonaws.com/amazon-eks/1.31.0/2024-09-12/bin/linux/amd64/kubect1
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 53.7M  100 53.7M    0     0  9511k      0  0:00:05  0:00:05 --:--:-- 10.2M
[ec2-user@ip-172-31-25-198 manifests]$ sudo chmod +x /usr/local/bin/kubect1
[ec2-user@ip-172-31-25-198 manifests]$ kubect1 version
Client Version: v1.31.0-eks-a737599
Kustomize Version: v5.4.2
Server Version: v1.33.6-eks-5afda5c
WARNING: version difference between client (1.31) and server (1.33) exceeds the supported minor version skew of +/-1
[ec2-user@ip-172-31-25-198 manifests]$ kubect1 apply -f flask-deployment.yaml
deployment.apps/nextwork-flask-backend created
[ec2-user@ip-172-31-25-198 manifests]$
```



Verifying Deployment

My extension for this project is to use the EKS console to visually monitor my Kubernetes deployment, including checking the status of Pods, nodes, and Services, and verifying that my backend application is running correctly inside the cluster. I had to set up IAM access policies because AWS requires explicit permission for an IAM user to access and interact with the Kubernetes control plane through the EKS console. Without these permissions, I would not be able to view or manage cluster resources. I set up access by creating an IAM access entry for my user, attaching the AmazonEKSClusterAdminPolicy, and granting cluster-level access so I could fully view and manage all Kubernetes resources through the AWS EKS console.

Pods are the smallest deployable units in Kubernetes and represent one or more containers that run together on the same node. Each pod acts as a single instance of an application, which Kubernetes creates, scales, and replaces as needed. Containers in a pod share the same network namespace and storage, meaning they use the same IP address, ports, and can access the same volumes. This allows containers within a pod to communicate easily and work together as a single application unit.

The EKS console shows the events for each pod, where I could see Kubernetes scheduling the pod onto a node, pulling the container image from Amazon ECR, creating the container, and starting it successfully. This validated that my Deployment was working correctly — Kubernetes was able to retrieve the backend image, launch the container, and run the application without errors, confirming that the backend was successfully deployed and operational in the EKS cluster.



Events (5)

Type	Reason	Event time	From	Message	🔍 Inspect with Amazon Q
🟢 Normal	Scheduled	10 minutes ago	default-scheduler	Successfully assigned default/nextwork-flask-backend-6fc85d47c8-flghp to ip-192-168-60-235.ec2.internal	-
🟢 Normal	Pulling	10 minutes ago	kubelet	Pulling image "006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest"	-
🟢 Normal	Pulled	9 minutes ago	kubelet	Successfully pulled image "006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest" in 2.787s (2.787s including waiting). Image size: 39996297 bytes.	-
🟢 Normal	Created	9 minutes ago	kubelet	Created container: nextwork-flask-backend	-
🟢 Normal	Started	9 minutes ago	kubelet	Started container nextwork-flask-backend	-



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

